

Food Delivery Operations & Performance Analysis

Data Analyst Project | Python • SQL • Power BI

Project Report

45,584 Total Orders	1,562 Delayed Orders	3.43% Delay Rate
44,022 Timely Orders	26.29 min Avg. Delivery Time	4.64 Avg. Delivery Rating

Project Summary

This project analyzes food-delivery order and delivery-operations data to understand delivery efficiency and the operational factors associated with longer delivery times. Python was used for data cleaning and exploratory analysis, SQL for business-oriented querying, and Power BI for interactive reporting.

Contents

- 1. Project Overview
- 2. Dataset & Data Preparation
- 3. Python Data Cleaning Script
- 4. Exploratory Data Analysis (EDA)
- 5. SQL Analysis
- 6. Power BI Dashboard
- 7. Consolidated Business Insights
- 8. Business Recommendations
- 9. Data Quality & Limitations
- 10. Analyst Methodology & Skills
- 11. Project Explanation
- 12. Appendix A — Full Python EDA Script

Portfolio Positioning

The project demonstrates an end-to-end analytics workflow: preparing messy operational data, exploring patterns with Python, answering business questions with SQL, and management-friendly Power BI dashboard.

1. Project Overview

The objective of this project is to analyze food-delivery operations and identify the conditions that influence delivery time. The analysis focuses on traffic, weather, multiple deliveries, vehicle type, city, delivery-person ratings, festival status, and order hour.

Business Questions

- What is the overall distribution of delivery time?
- How does traffic density affect delivery time?
- How does weather affect delivery time?
- How do multiple deliveries affect delivery time?
- How does delivery time vary by vehicle type?
- How does delivery time vary by city?
- How does delivery time vary by delivery-person rating?
- How does festival status affect delivery time?
- How does delivery time vary by order hour?

Technology Stack

- Python: Pandas, NumPy, Matplotlib, Seaborn
- SQL / MySQL: GROUP BY, HAVING, CASE, CTEs, joins, window functions, date functions
- Power BI: KPI cards, trend charts, bar charts, slicers, and combination charts

Project Workflow

End-to-End Workflow

Raw data → Python data cleaning → Cleaned CSV → SQL business analysis → Python EDA → Power BI dashboard
→ Key insights → Business recommendations

2. Dataset & Data Preparation

The cleaned dataset contains 45,584 order records and 1,320 unique delivery persons. The dataset includes order identifiers, delivery-person information, location coordinates, date/time fields, weather, traffic, vehicle information, festival status, city, and delivery duration.

Main Fields

Category	Fields
Order & Person	id, delivery_person_id, delivery_person_age, delivery_person_ratings
Location	restaurant_latitude, restaurant_longitude, delivery_location_latitude, delivery_location_longitude, city
Time	order_date, time_orderd, time_order_picked, time_taken_min
Operations	weather_conditions, road_traffic_density, vehicle_condition, multiple_deliveries, festival
Vehicle / Order	type_of_order, type_of_vehicle

Cleaning & Preparation Steps

- Loaded the raw CSV with Pandas and inspected structure, shape, and data types.
- Standardized column names to lowercase and renamed time_taken (min) to time_taken_min.
- Converted mixed-format time fields into a consistent representation.
- Standardized pickup times beginning with 24: to 00: where required.
- Filled missing categorical values such as weather, traffic, festival, and city with Unknown/No as appropriate.
- Filled missing multiple-delivery values with 0.
- Converted multiple_deliveries to integer.
- Converted order_date to datetime.
- Handled invalid delivery-person ages below 18 using missing-value replacement followed by the median.
- Handled delivery-person ratings outside the 1–5 range and filled missing ratings with the median.
- Created a delivery-time clock field from pickup time plus time_taken_min.
- Exported the cleaned dataset as food_delivery_cleaned.csv.

Key Insight

The cleaning process preserves the original order records while standardizing fields needed for SQL, Python EDA, and Power BI. Missing categories are represented explicitly rather than silently deleting records.

3. Python Data Cleaning Script

The following is the actual cleaning script used for the project. It is included so the portfolio report documents the transformation logic rather than only presenting final charts.

```
import pandas as pd
import numpy as np

df=pd.read_csv('food_delivery.csv')
print(df.head(5))
df.info()
print(df.shape)
df.columns=df.columns.str.strip().str.lower()
df.rename(columns={'time_taken (min)': 'time_taken_min'}, inplace=True)

def convert_time(x):
    if pd.isna(x):
        return x

    x = str(x).strip()

    # Decimal time value
    if ':' not in x:
        try:
            seconds = round(float(x) * 24 * 60 * 60)
            return (pd.Timestamp('2000-01-01') +
                    pd.Timedelta(seconds=seconds)).strftime('%H:%M')
        except ValueError:
            return x

    # Already in HH:MM format
    return x

df['time_orderd'] = df['time_orderd'].apply(convert_time)

df['time_order_picked'] = df['time_order_picked'].apply(convert_time)

# Clean pickup time
df['time_order_picked'] = (
    df['time_order_picked']
    .astype(str)
    .str.strip()
    .str.replace(r'^24:', '00:', regex=True)
)

df['time_orderd'] = df['time_orderd'].fillna(
    df['time_orderd'].mode()[0]
)

df['weather_conditions'] = df['weather_conditions'].fillna('Unknown')

df['road_traffic_density'] = df['road_traffic_density'].fillna('Unknown')

df['multiple_deliveries'] = df['multiple_deliveries'].fillna(0)

df['festival'] = df['festival'].fillna('No')

df['city'] = df['city'].fillna('Unknown')

print(df.duplicated().sum())
print(df['id'].duplicated().sum())

df['multiple_deliveries'] = df['multiple_deliveries'].astype(int)
df['order_date'] = pd.to_datetime(
```

```

    df['order_date'],
    format='%d-%m-%Y',
    errors='coerce'
)

df['time_order_picked'] = df['time_order_picked'].fillna(
    df['time_order_picked'].mode()[0]
)

print(df['delivery_person_age'].describe())

df.loc[df['delivery_person_age'] < 18, 'delivery_person_age'] = np.nan

df['delivery_person_age'] = df['delivery_person_age'].fillna(
    df['delivery_person_age'].median()
)
df['delivery_person_age'] = df['delivery_person_age'].astype(int)

print(df['delivery_person_ratings'].describe())
print("Ratings above 5:", (df['delivery_person_ratings'] > 5).sum())
df.loc[
    (df['delivery_person_ratings'] < 1) |
    (df['delivery_person_ratings'] > 5),
    'delivery_person_ratings'
] = np.nan
df['delivery_person_ratings'] = df['delivery_person_ratings'].fillna(
    df['delivery_person_ratings'].median()
)

print(df['time_taken_min'].describe())

print(df['delivery_person_age'].describe())
print(df['delivery_person_ratings'].describe())

df['delivery_time'] = (
    pd.to_datetime(df['time_order_picked'], format='mixed', errors='coerce')
    + pd.to_timedelta(df['time_taken_min'], unit='min')
).dt.strftime('%H:%M')

print(df.isnull().sum())
print(df.dtypes)
print(df.shape)
print(df['delivery_time'].head(5))

df.to_csv('food_delivery_cleaned.csv', index=False)

```

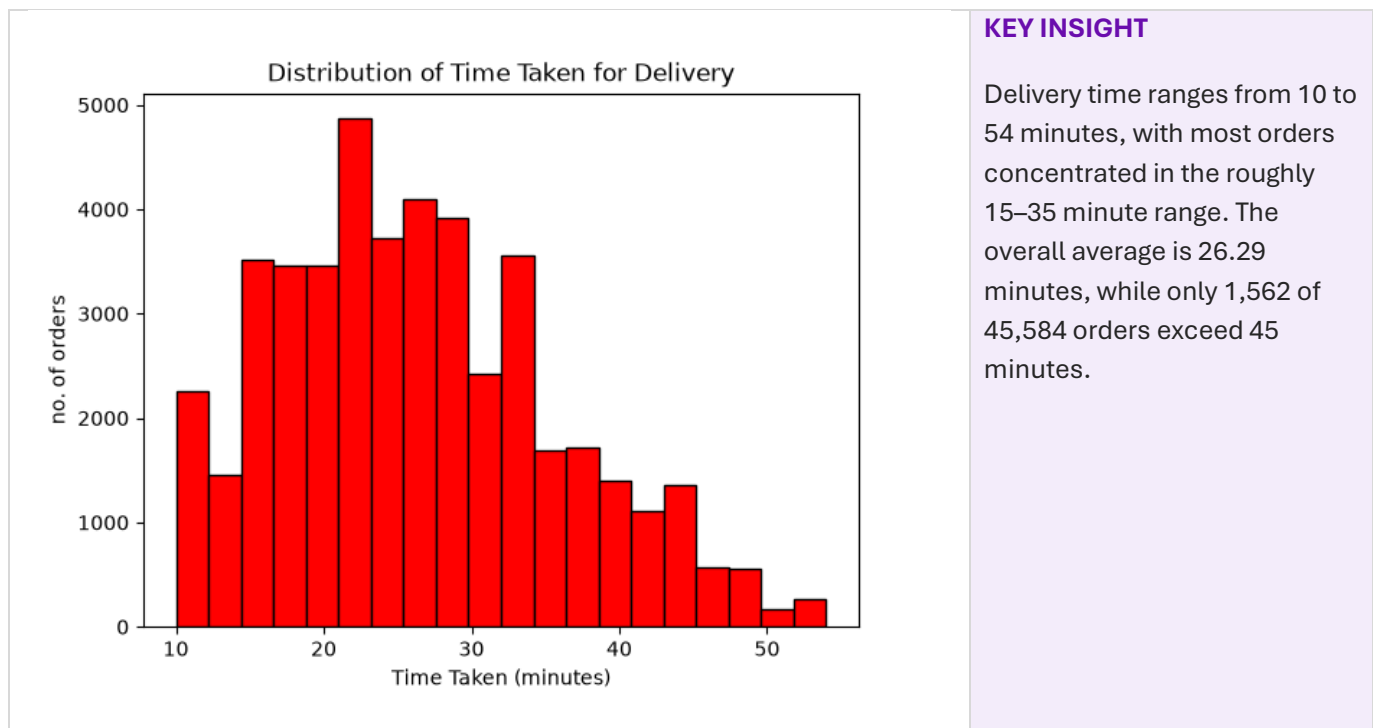
4. Exploratory Data Analysis (EDA)

Python EDA was used to explore delivery-time patterns before presenting the final findings in Power BI. The analysis uses straightforward Pandas groupby/aggregation operations and Matplotlib charts so each result can be explained clearly in a Data Analyst interview.

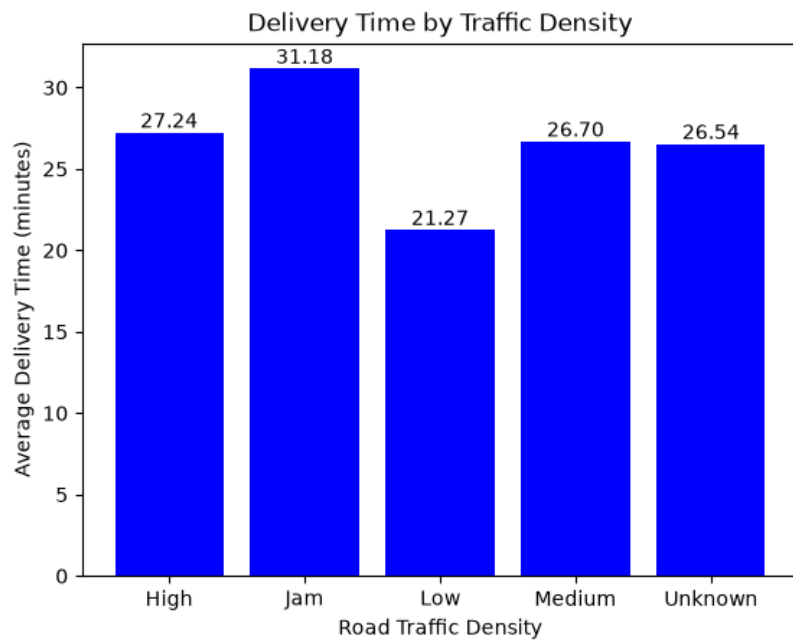
EDA Approach

- Loaded the cleaned CSV.
- Converted date/time fields for analysis.
- Calculated average delivery time by operational dimensions.
- Used a histogram for the distribution of delivery time.
- Used bar charts for categorical comparisons.
- Used line charts for hourly and rating-based patterns.
- Added data labels to categorical bar charts for easier interpretation.

Q1 — Distribution of Delivery Time



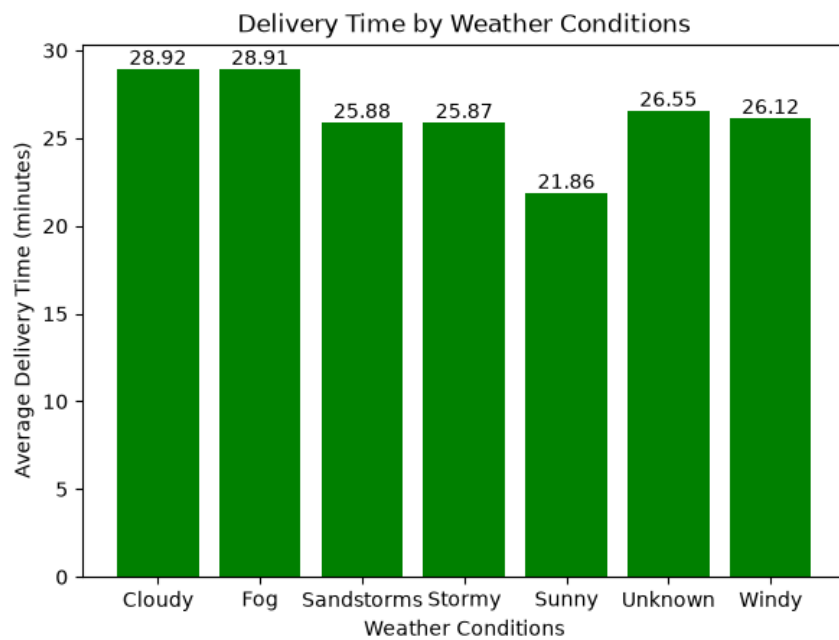
Q2 — Delivery Time by Traffic Density



KEY INSIGHT

Jam traffic has the highest average delivery time at 31.18 minutes, while Low traffic has the lowest at 21.27 minutes. This shows a clear operational relationship between road congestion and delivery time.

Q3 — Delivery Time by Weather Conditions



KEY INSIGHT

Cloudy and Fog conditions have the highest average delivery times at 28.92 and 28.91 minutes respectively. Sunny conditions are the lowest at 21.86 minutes, indicating that weather conditions can contribute to delivery delays.

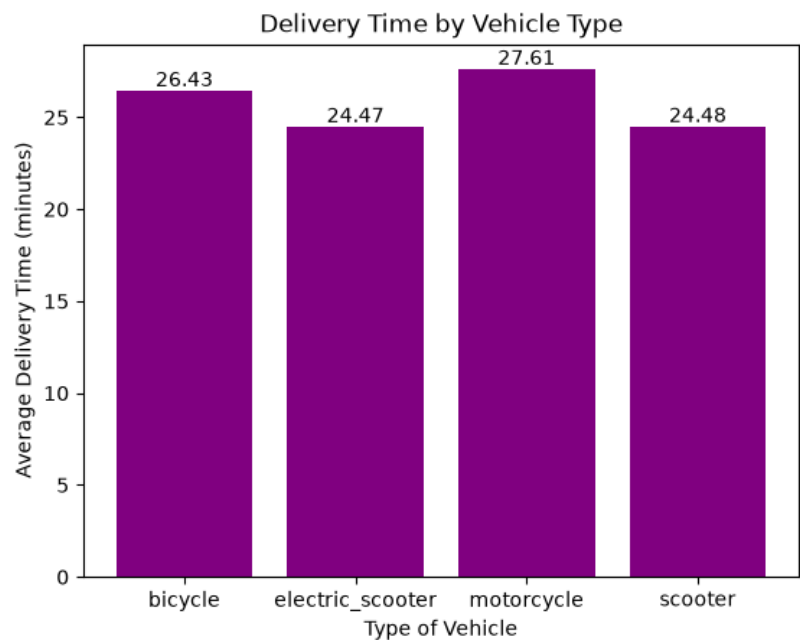
Q4 — Delivery Time by Multiple Deliveries



KEY INSIGHT

Average delivery time rises as the number of simultaneous deliveries increases: 22.87 minutes for 0, 26.85 for 1, 40.45 for 2, and 47.82 for 3. This is one of the clearest operational drivers in the dataset.

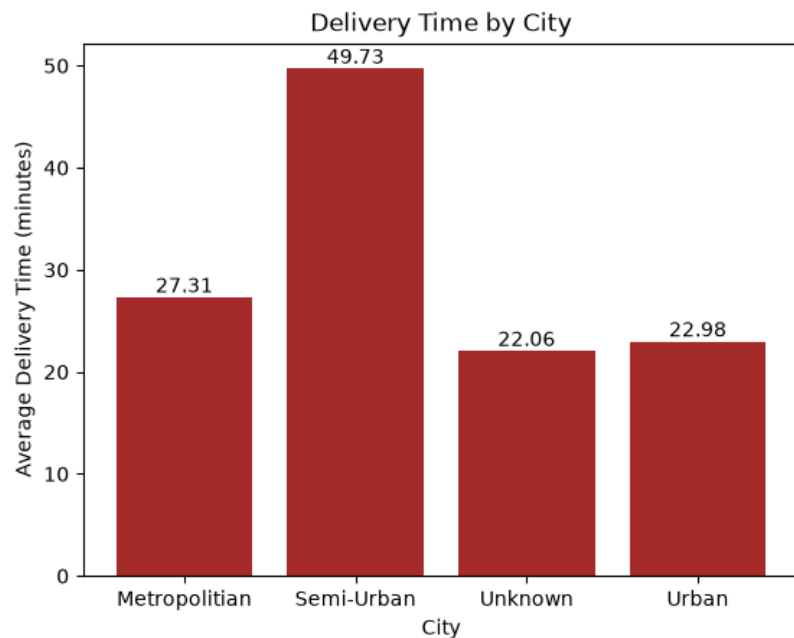
Q5 — Delivery Time by Vehicle Type



KEY INSIGHT

Motorcycles have the highest average delivery time at 27.61 minutes, while electric scooters are lowest at 24.47 minutes. Bicycle shows 26.43 minutes but represents only 68 orders, so it was excluded from the Power BI comparison chart for readability and sufficient-volume comparison.

Q6 — Delivery Time by City



KEY INSIGHT

Semi-Urban has the highest average delivery time at 49.73 minutes, but only 164 orders are in this category. Metropolitan averages 27.31 minutes, Urban 22.98 minutes, and Unknown 22.06 minutes. The very small Semi-Urban volume should be considered when interpreting the result.

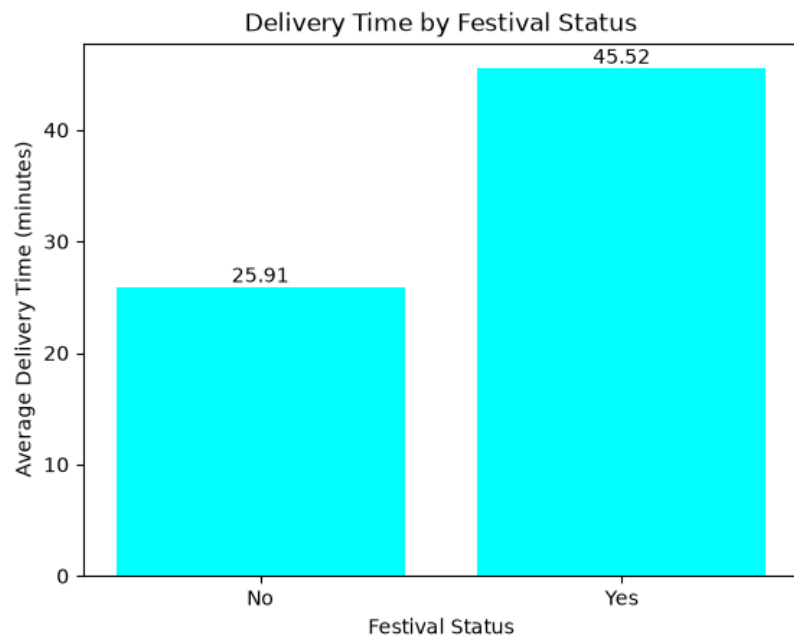
Q7 — Delivery Time by Delivery-Person Rating



KEY INSIGHT

The chart shows a noticeable change in delivery time across rating bands, with higher-rating groups generally associated with lower average delivery times. However, several low-rating bands have very small order counts, so the pattern should be treated as directional rather than causal.

Q8 — Delivery Time by Festival Status



KEY INSIGHT

Festival orders average 45.52 minutes compared with 25.91 minutes for non-festival orders. The large difference suggests that festival periods are associated with significantly higher delivery pressure.

Q9 — Delivery Time by Order Hour



KEY INSIGHT

Average delivery time is lowest around 10:00 at 19.48 minutes and peaks around 20:00 at 31.19 minutes. Evening hours therefore show substantially higher delivery times than the late-morning period.

5. SQL Analysis

SQL was used to answer focused business questions from the cleaned data. The analysis combines aggregation with selected CTE, ranking, comparison, and window-function techniques. The goal is to demonstrate practical SQL rather than unnecessary complexity.

Q1. Top 3 delivery persons in each city based on average delivery performance, considering only delivery persons with a minimum number of orders?

```
WITH delivery_person_rnk AS
(
SELECT delivery_person_id, city,
       ROUND(AVG(delivery_person_rating),2) as avg_performance,
       DENSE_RANK() OVER(PARTITION BY city ORDER BY AVG(delivery_person_rating) DESC) AS dr
FROM zomato
GROUP BY delivery_person_id, city
HAVING count(*) >= 5
)
SELECT delivery_person_id, avg_performance,city, dr
FROM delivery_person_rnk
WHERE dr <= 3
order by city, dr;
```

delivery_person_id	avg_performance	city	dr
ALHRES02DEL02	4.90	Metropolitan	1
LUDHRES20DEL03	4.85	Metropolitan	2
GOARES14DEL01	4.85	Metropolitan	2
AGRRES08DEL01	4.84	Metropolitan	3
AGRRES19DEL02	4.84	Metropolitan	3
BANGRES06DEL02	4.88	Unknown	1
MUMRES18DEL01	4.85	Unknown	2
VADRES17DEL01	4.80	Unknown	3
HYDRES04DEL03	4.92	Urban	1
CHENRES09DEL03	4.89	Urban	2
DEHRES03DEL01	4.87	Urban	3

Q2. Which delivery persons show the greatest improvement in delivery performance from one month to the next?

```
WITH performance_change AS
(SELECT delivery_person_id, DATE_FORMAT(order_date, '%Y-%m') AS month,
       ROUND(LAG(AVG(delivery_person_rating)) OVER(PARTITION BY delivery_person_id
       ORDER BY DATE_FORMAT(order_date, '%Y-%m')),2) AS previous_performance,
       ROUND(AVG(delivery_person_rating),2) AS current_performance
FROM zomato
GROUP BY delivery_person_id, DATE_FORMAT(order_date, '%Y-%m')
)
SELECT delivery_person_id, month, previous_performance, current_performance,
       current_performance - previous_performance AS improvement
FROM performance_change
WHERE previous_performance IS NOT NULL
ORDER BY improvement DESC
LIMIT 1;
```

delivery_person_id	month	previous_performance	current_performance	improvement
HYDRES14DEL03	2022-04	4.46	4.80	0.34

Q3. Highest and lowest average delivery-time cities and how many orders does each city handle?

```
(SELECT city , COUNT(*) AS total_orders,
       AVG(time_taken_min) AS avg_delivery_time
FROM zomato
GROUP BY city
ORDER BY avg_delivery_time DESC
LIMIT 1)
UNION ALL
(SELECT city , COUNT(*) AS total_orders,
       AVG(time_taken_min) AS avg_delivery_time
FROM zomato
GROUP BY city
ORDER BY avg_delivery_time ASC
LIMIT 1);
```

city	total_orders	avg_delivery_time
Semi-Urban	164	49.7317
Unknown	1200	22.0583

Q4. Percentage of delayed orders by city (delivery time > 45 minutes) in each city?

```
SELECT city,  
       ROUND(SUM(CASE WHEN time_taken_min > 45 THEN 1 END * 100.0) /  
             COUNT(*),2) AS delayed_percentage  
FROM zomato  
GROUP BY city;
```

city	delayed_percentage
Urban	1.58
Metropolitan	3.62
Unknown	0.83
Semi-Urban	96.95

Q5. Best delivery person with at least 50 orders?

```
SELECT delivery_person_id,  
       COUNT(*) AS total_orders,  
       ROUND(AVG(delivery_person_rating),2) AS delivery_person_performance  
FROM zomato  
GROUP BY delivery_person_id  
HAVING COUNT(*) >= 50  
ORDER BY delivery_person_performance DESC  
LIMIT 1;
```

delivery_person_id	total_orders	delivery_person_performance
VADRES08DEL01	51	4.78

Q6. Traffic + weather combination with longest average delivery time?

```
SELECT  
       road_traffic_density,  
       weather_conditions,  
       AVG(time_taken_min) AS avg_delivery_time  
FROM zomato  
GROUP BY road_traffic_density, weather_conditions  
ORDER BY avg_delivery_time DESC  
LIMIT 1;
```

road_traffic_density	weather_conditions	avg_delivery_time
High	Unknown	49.0000

Q7. How does delivery performance change month-over-month, and which month shows the biggest increase or decrease in average delivery time?

```
WITH previous_time AS
(SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
      AVG(delivery_person_rating) AS delivery_person_performance,
      AVG(time_taken_min) AS current_avg_delivery_time
FROM zomato
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
),
time_diff AS
(SELECT month, delivery_person_performance,
      current_avg_delivery_time,
      LAG(current_avg_delivery_time) OVER (ORDER BY month) AS previous_delivery_time
FROM previous_time)
SELECT month, delivery_person_performance,
      previous_delivery_time,
      current_avg_delivery_time,
      (current_avg_delivery_time - previous_delivery_time) AS diff_delivery_time
FROM time_diff
WHERE previous_delivery_time IS NOT NULL;
```

month	delivery_person_performance	previous_delivery_time	current_avg_delivery_time	diff_delivery_time
2022-03	4.63418	26.5331	26.2754	-0.2577
2022-04	4.63251	26.2754	26.1152	-0.1602

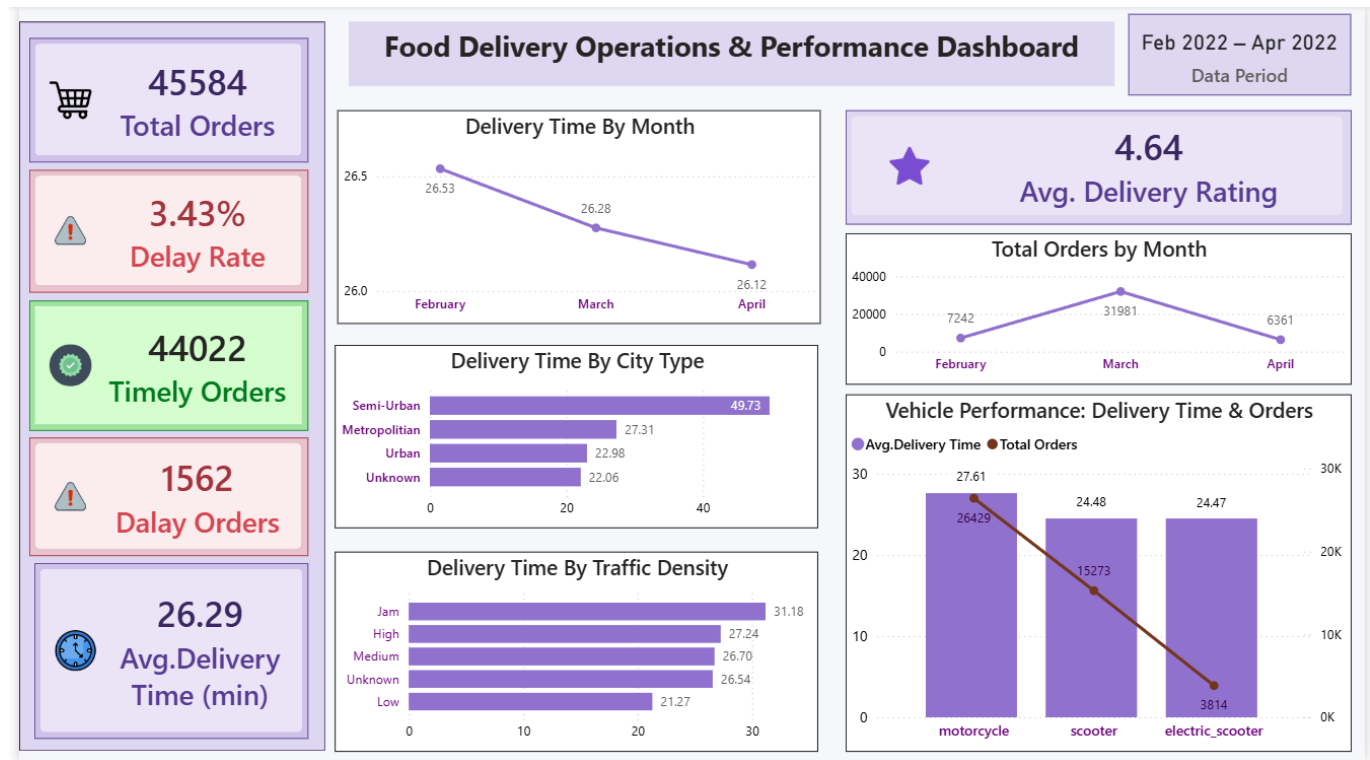
Q8. Best vehicle type + condition provide best performance with at least 100 orders?

```
SELECT
      type_of_vehicle,
      vehicle_condition,
      COUNT(*) AS total_orders,
      ROUND(AVG(time_taken_min), 2) AS delivery_time
FROM zomato
GROUP BY type_of_vehicle, vehicle_condition
HAVING COUNT(*) >= 100
ORDER BY delivery_time ASC
LIMIT 1;
```

type_of_vehicle	vehicle_condition	total_orders	delivery_time
motorcycle	1	7583	24.30

6. Power BI Dashboard

The Power BI dashboard translates the analytical results into an interactive management-style view. The final design emphasizes operational KPIs, time trends, city performance, traffic impact, and vehicle performance.



Key Insight

The dashboard provides a concise operational view: 45,584 total orders, 1,562 delayed orders, a 3.43% delay rate, 44,022 timely orders, 26.29 minutes average delivery time, and 4.64 average delivery rating.

Dashboard Visuals & Purpose

- KPI Cards: summarize total orders, delayed orders, delay rate, timely orders, average delivery time, and average rating.
- Delivery Time by Month: shows the month-level movement in average delivery time.
- Total Orders by Month: shows order-volume movement across the observed months.
- Delivery Time by City: compares operational performance across city categories.
- Delivery Time by Traffic Density: highlights the effect of congestion.
- Vehicle Performance: combines average delivery time with order volume for the higher-volume vehicle categories.

Dashboard Design Decision

Vehicle Category Filtering

Bicycle remains in the underlying dataset and is available for analysis. It was filtered only from the combined vehicle-performance visual because it has 68 orders, far below the volume of the other vehicle categories. This prevents a very small category from dominating the visual scale and improves readability.

7. Consolidated Business Insights

Area	Key Insight
Overall performance	The dataset contains 45,584 orders with an average delivery time of 26.29 minutes and an overall delivery-person rating of 4.64.
Delay performance	1,562 orders take more than 45 minutes, representing a 3.43% delay rate.
Traffic impact	Jam traffic averages 31.18 minutes versus 21.27 minutes under Low traffic.
Multiple deliveries	Average delivery time increases from 22.87 minutes with no multiple deliveries to 47.82 minutes with three.
Festival impact	Festival orders average 45.52 minutes versus 25.91 minutes for non-festival orders.
Weather	Cloudy (28.92) and Fog (28.91) have the highest average delivery times, while Sunny is lowest at 21.86 minutes.
City	Semi-Urban records 49.73 minutes on average, but only 164 orders are in that category; Metropolitan is 27.31 minutes and Urban is 22.98.
Vehicle	Motorcycle averages 27.61 minutes, compared with 24.47 for electric scooter.
Peak hour	Average delivery time peaks around 20:00 at 31.19 minutes and is lowest around 10:00 at 19.48 minutes.

8. Business Recommendations

Reduce congestion-driven delays — Use peak-period staffing and delivery allocation to reduce the impact of Jam and High traffic conditions.

Control multiple-delivery load — Review batching rules because delivery time rises substantially when a delivery person handles two or three simultaneous deliveries.

Plan for festival demand — Festival periods show a much higher average delivery time; capacity and delivery-person availability should be increased during these periods.

Investigate high-risk city segments — Semi-Urban has the highest average delivery time and delay rate, but its low volume means the finding should be validated before major operational changes.

Use vehicle performance carefully — Compare vehicle efficiency together with order volume and vehicle condition rather than using average delivery time alone.

Monitor evening operations — Average delivery time rises sharply in the evening, especially around 19:00–21:00, suggesting a need for stronger peak-hour capacity.

Improve data quality monitoring — Continue tracking missing city, weather, and traffic values so operational reports distinguish true business patterns from data-quality effects.

9. Data Quality & Limitations

- City, weather, traffic, and other categorical fields contain Unknown values after cleaning; these are retained to avoid unnecessary row deletion.
- Some categories have very small volumes. For example, Bicycle has only 68 orders and Semi-Urban has 164 orders. Such averages should be interpreted with sample size in mind.
- The delivery-person rating analysis contains low-rating groups with small order counts, so the visual should not be interpreted as proof that rating causes delivery speed.
- The observed order dates cover February–April in the current cleaned dataset, so month-of-year seasonality should not be generalized to a full year.
- The project is an analytical portfolio exercise; recommendations are based on the available dataset and should be validated with operational and business context before implementation.

Analytical Principle

A high or low average is not enough by itself. Order volume, missing values, and category context should be considered before turning an observed pattern into a business decision.

10. Analyst Methodology & Skills

Methodology

- Define the operational problem and relevant KPIs.
- Inspect the raw dataset and identify data-quality issues.
- Clean and standardize fields with Python/Pandas.
- Create a reusable cleaned dataset.
- Explore delivery-time patterns using grouped statistics and visualizations.
- Use SQL to answer targeted business questions and compare operational segments.
- Build a Power BI dashboard for interactive reporting.
- Translate analytical results into concise insights and practical recommendations.

Skills Demonstrated

- Python: Pandas, NumPy, datetime handling, missing-value treatment, validation, groupby and aggregation.
- Data Visualization: Matplotlib and Seaborn-based exploratory charts.
- SQL: GROUP BY, HAVING, CASE, UNION ALL, CTEs, joins, LAG, DENSE_RANK, date functions.
- Power BI: KPI cards, measures, slicers, categorical comparisons, trend charts, combo charts, formatting.
- Business Analysis: KPI interpretation, operational-driver analysis, sample-size awareness, and recommendations.

11. Project Explanation

Interview-Ready Explanation

I worked on a food delivery operations and performance analysis project using 45,584 order records. I first used Python and Pandas to inspect and clean the data, including standardizing time fields, handling missing categorical values, validating age and rating ranges, and creating a delivery-time field. Then I used SQL to answer business questions around city performance, delayed orders, delivery-person performance, traffic and weather combinations, month-over-month delivery time, and vehicle performance. Finally, I built a Power BI dashboard with KPI cards and operational visuals. The main findings were that congestion, multiple deliveries, festival periods, and evening hours were associated with longer delivery times. I translated those findings into recommendations around peak-hour capacity, batching, festival planning, and operational monitoring.

12. Appendix A — Full Python EDA Script

Full script used to generate the nine EDA analyses shown in Section 4.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('food_delivery_cleaned.csv')

df['order_date'] = pd.to_datetime(
    df['order_date'],
    errors='coerce'
)

df['time_orderd'] = pd.to_datetime(
    df['time_orderd'],
    format='mixed',
    errors='coerce'
)

df['time_order_picked'] = pd.to_datetime(
    df['time_order_picked'],
    format='mixed',
    errors='coerce'
)

print(df.dtypes)
print(df[['order_date', 'time_orderd', 'time_order_picked']].isna().sum())

#Analysis:

# Q1. What is the distribution of delivery time?
plt.hist(df['time_taken_min'], bins = 20, color = 'red', edgecolor = 'black')
plt.xlabel('Time Taken (minutes)')
plt.ylabel('no. of orders')
plt.title('Distribution of Time Taken for Delivery')
plt.show()

# # Q2.How does delivery time vary by traffic density?
avg_delivery_time = df.groupby('road_traffic_density')['time_taken_min'].mean()

plt.bar(avg_delivery_time.index, avg_delivery_time.values, color = 'blue')
plt.bar_label(plt.bar(avg_delivery_time.index, avg_delivery_time.values, color = 'blue'), fmt='%.2f')
plt.xlabel('Road Traffic Density')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Traffic Density')
plt.show()

# # Q3. How does weather affect delivery time?
avg_delivery_time_by_weather = df.groupby('weather_conditions')['time_taken_min'].mean()

plt.bar(avg_delivery_time_by_weather.index, avg_delivery_time_by_weather.values, color = 'green')
plt.bar_label(plt.bar(avg_delivery_time_by_weather.index, avg_delivery_time_by_weather.values, color = 'green'), fmt = '%.2f')
plt.xlabel('Weather Conditions')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Weather Conditions')
plt.show()

# # Q4. How do multiple deliveries affect delivery time?
avg_delivery_time_by_multiple_deliveries = df.groupby('multiple_deliveries')['time_taken_min'].mean()

plt.bar(avg_delivery_time_by_multiple_deliveries.index, avg_delivery_time_by_multiple_deliveries.values, color= 'orange')
```

```

plt.bar_label(plt.bar(avg_delivery_time_by_multiple_deliveries.index, avg_delivery_time_by_multiple_deliveries.values,
color= 'orange'), fmt= '%.2f')
plt.xlabel('Multiple Deliveries')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Multiple Deliveries')
plt.show()

# # Q5. How does delivery time vary by vehicle type?
avg_delivery_time_by_vehicle_type = df.groupby('type_of_vehicle')['time_taken_min'].mean()

plt.bar(avg_delivery_time_by_vehicle_type.index, avg_delivery_time_by_vehicle_type.values, color = 'purple')
plt.bar_label(plt.bar(avg_delivery_time_by_vehicle_type.index, avg_delivery_time_by_vehicle_type.values, color =
'purple'), fmt = '%.2f')
plt.xlabel('Type of Vehicle')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Vehicle Type')
plt.show()

# # Q6. How does delivery time vary by city?
avg_delivery_time_by_city = df.groupby('city')['time_taken_min'].mean()

plt.bar(avg_delivery_time_by_city.index, avg_delivery_time_by_city.values, color = 'brown')
plt.bar_label(plt.bar(avg_delivery_time_by_city.index, avg_delivery_time_by_city.values, color = 'brown'), fmt = '%.2f')
plt.xlabel('City')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by City')
plt.show()

# Q7. How does delivery time vary by delivery-person rating?
avg_delivery_time_by_delivery_person_rating = df.groupby('delivery_person_ratings')['time_taken_min'].mean()

plt.plot(avg_delivery_time_by_delivery_person_rating.index, avg_delivery_time_by_delivery_person_rating.values, color =
'pink', marker = 'o')
plt.xlabel('Delivery Person Rating')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Delivery Person Rating')
plt.show()

# Q8. How does delivery time vary by festival status?
avg_delivery_time_by_festival = df.groupby('festival')['time_taken_min'].mean()

plt.bar(avg_delivery_time_by_festival.index, avg_delivery_time_by_festival.values, color = 'cyan')
plt.bar_label(plt.bar(avg_delivery_time_by_festival.index, avg_delivery_time_by_festival.values, color = 'cyan'), fmt =
'%.2f')
plt.xlabel('Festival Status')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Festival Status')
plt.show()

# Q9. How does delivery time vary by order hour?
df['order_hour'] = df['time_orderd'].dt.hour
avg_delivery_time_by_order_hour = df.groupby('order_hour')['time_taken_min'].mean()

plt.plot(avg_delivery_time_by_order_hour.index, avg_delivery_time_by_order_hour.values, color = 'magenta', marker = 'o')
plt.xlabel('Order Hour')
plt.ylabel('Average Delivery Time (minutes)')
plt.title('Delivery Time by Order Hour')
plt.show()

```